

----- Slip 1 -----

Q.1) Take multiple files as Command Line Arguments and print their inode numbers and file types

```
#include <stdio.h>

#include <sys/stat.h>

void printFileType(mode_t mode) {
    if (S_ISREG(mode))    printf("Regular File");
    else if (S_ISDIR(mode)) printf("Directory");
    else if (S_ISCHR(mode)) printf("Character Device");
    else if (S_ISBLK(mode)) printf("Block Device");
    else if (S_ISFIFO(mode)) printf("FIFO/Named Pipe");
    else if (S_ISLNK(mode)) printf("Symbolic Link");
    else if (S_ISSOCK(mode)) printf("Socket");
    else
        printf("Unknown Type");
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <file1> <file2> ...\n", argv[0]);
        return 1;
    }
    struct stat fileStat;
    for (int i = 1; i < argc; i++) {
        if (stat(argv[i], &fileStat) == -1) {
            perror(argv[i]);
            continue;
        }
        printf("\nFile: %s\n", argv[i]);
        printf("Inode Number: %lu\n", fileStat.st_ino);
        printf("File Type: ");
        printFileType(fileStat.st_mode);
        printf("\n");
    }

    return 0;
}
```

Q.2) Write a C program to send SIGALRM signal by child process to parent process and parent process

make a provision to catch the signal and display alarm is fired.(Use Kill, fork, signal and sleep

system call)

```
#include <stdio.h>

#include <unistd.h>

#include <signal.h>

#include <stdlib.h>

// Signal handler for parent
void alarm_handler(int sig) {
    if (sig == SIGALRM) {
        printf("Alarm is fired!\n");
    }
}

int main() {
    pid_t pid;

    // Parent sets up signal handler
    signal(SIGALRM, alarm_handler);

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child process
        sleep(2); // Give parent time to set handler
        printf("Child sending SIGALRM to parent...\n");
        kill(getppid(), SIGALRM);
    } else {
        // Parent process
        printf("Parent waiting for alarm signal...\n");

        // Parent waits for signal
        pause(); // suspend until signal is received

        printf("Parent received the signal. Exiting.\n");
    }
}
```

```
return 0;  
}
```